# The Comonadic Sweep Harness

Parameter tuning as a Store comonad, and why that is the right shape

Economy showpiece — `harness-worker-orchestration/economy`

6 November 2026

**Abstract**

The economy showpiece is tuned by a *sweep harness*: a program that runs the simulated town under many parameter vectors and reports what changed. This note shows that the harness is not merely organised with a categorical flavour — it *is* the **Store comonad**, exactly. The comonad's three operations ($\varepsilon$, $\delta$, extend) turn out to be precisely the primitives a tuning tool needs: run the baseline, read a counterfactual, and compute a *neighbourhood-aware* quantity such as a sensitivity gradient. We derive the sensitivity analysis as a finite-difference kernel under extend, show that the comonad laws are a correctness contract on the instrument, and prove that the simulation layer (a State monad) and the tuning layer (a Store comonad) are the monad and comonad of *one and the same adjunction*. The emphasis throughout is on what the structure *buys you*: counterfactuals that cannot corrupt your baseline, a grid and a gradient from the same operator, higher-order cross-elasticities that are guaranteed coherent, and a tuning calculus completely decoupled from the model it tunes.

## 1 The harness *is* a Store

The Store comonad (also Costate, the "spreadsheet" comonad) is a readout from positions to values, paired with a distinguished position — the *focus*:

$$\mathsf{Store}\, S\, A \;=\; (S \to A) \times S.$$

In code:

```
export type Store<S, A> = { readonly peek: (s: S) => A; readonly pos: S };
```

The harness instantiates this with no encoding step whatsoever:

| Store component | Type | In the harness |
|---|---|---|
| position | $S$ | a full `Params` vector |
| readout | $S \to A$ | "boot the town at these params, run $N$ months, summarise" |
| focus | $S$ | the baseline `Params` |
| value | $A$ | a `Result` (deaths, folds, prices, cash band) |

The readout is a genuine *function* because the economy runs under a seeded RNG and a fixed clock: the same parameters always produce the same `Result`. That determinism is the hinge on which everything below turns.

## 2 The operations are tuning primitives

A comonad provides $\varepsilon : WA \to A$ and $\delta : WA \to W(WA)$, from which extend is derived. For the Store comonad these specialise to a vocabulary that reads, position by position, like the things a person tuning a simulation actually does.

| Operation | Type | What it *is* when tuning |
|---|---|---|
| `extract` | $\mathsf{Store}\, S\, A \to A$ | run at the baseline: the number you are staring at |
| `peek` $s'$ | $S \to \mathsf{Store}\, S\, A \to A$ | a counterfactual, *without* moving the baseline |
| `seek` $s'$ | $S \to \mathsf{Store}\, S\, A \to \mathsf{Store}\, S\, A$ | re-baseline: commit to a new operating point |
| `experiment` | $(S \to [S]) \to \mathsf{Store}\, S\, A \to [A]$ | the grid: a whole shape of counterfactuals |
| `duplicate` | $\mathsf{Store}\, S\, A \to \mathsf{Store}\, S\, (\mathsf{Store}\, S\, A)$ | the entire response surface as a field |
| `extend` $k$ | $(\mathsf{Store}\, S\, A \to B) \to \mathsf{Store}\, S\, A \to \mathsf{Store}\, S\, B$ | a neighbourhood-aware quantity at every point |

The quietly important pair is `peek` versus `seek`. `peek` asks "what if `firmFloat` were 60?" and returns an answer while leaving the focus exactly where it was; `seek` actually relocates. In a hand-rolled sweep you mutate a global config, forget to restore it, and your "baseline" silently drifts. The comonad makes *ask without moving* and *move* two different, differently-typed operations — the drift bug is unrepresentable.

## 3 `extend` is the derivative

This is the crux, and the reason the structure must be a comonad rather than a plain `Array.map`.

`map`($f$) transforms each readout value *pointwise*: $f$ sees one `Result` and nothing around it. It can never compute a slope, because a slope needs two neighbouring points.

`extend`($k$) hands the kernel $k$ the *entire store refocused at each position*, so $k$ may `peek` at neighbours and return a difference. A local sensitivity $\partial(\text{metric})/\partial(\text{knob})$ is exactly a forward-difference kernel:

```
function slope(axis, step, metric) {
  return (w) => {
    // read this point, then its neighbour one step up the axis
    const here = metric(extract(w));
    const up   = metric(peek(bump(w.pos, axis, step), w));
    return (up - here) / step;   // forward difference: d metric / d knob
  };
}
// the sensitivity at the baseline focus:
const k = slope("stapleReservePrice", 4, r => r.firmsAlive);
const dFirms = extract(extend(k, w));
```

**Why this is the canonical use of Store.** The textbook extend over a Store is a *cellular automaton*: Conway's Life is extend(*rule*) over a Store of the grid, where *rule* reads the eight neighbours of the focused cell. The tuning harness is the same machine on a different lattice: each "cell" is a parameter point, and the "rule" reads adjacent configurations to compute a gradient. Sensitivity analysis and cellular automata are *the same comonadic computation*. This is why, run on the live town, `extend` found that the reserve price, not the cash float, is the dominant lever on firm survival:

| knob | $\partial$ firms | $\partial$ fed |
|---|---|---|
| `firmFloat` | 0.125 | 0.000 |
| `stapleReservePrice` | **0.500** | −0.250 |
| `townWage` | 0.000 | −0.750 |

`map` is blind to the neighbourhood where that slope lives; `extend` is defined by it.

## 4 The laws are a correctness contract

The harness ships eight passing law tests. They are not decoration — each is a guarantee about the tuning instrument.

**Law 1** (Counit)**.** $\varepsilon \circ \delta = \mathrm{id}$, *and* $\mathrm{map}\,\varepsilon \circ \delta = \mathrm{id}$. *Equivalently* $\varepsilon(\mathrm{extend}\,k) = k$*: the sensitivity you read at the focus is exactly the kernel you asked for, with no leakage from the refocusing machinery.*

**Law 2** (Coassociativity)**.** $\delta \circ \delta = \mathrm{map}\,\delta \circ \delta$. *Consequently* $\mathrm{extend}(k) \circ \mathrm{extend}(g)$ *is associative, so a sensitivity of a sensitivity — the Hessian, the cross-elasticities — is well defined and independent of the order in which the knobs are differentiated. "Does raising* `firmFloat` *change how sensitive the town is to* `stapleReservePrice`*?" is a coherent question precisely because this law holds.*

The hinge that makes all of it legal is **determinism**: `peek` $s$ is a function only if the same params always give the same `Result`. The seeded, fixed-clock replay that the project's `verify.sh` already demands is *also* what makes the tuning calculus sound. Were the model nondeterministic, `peek` would not be a function and the gradients would be meaningless.

## 5 Simulation and tuning are one adjunction

The relationship between the two layers of the system is tighter than "monad inside, comonad outside." Fix an object $S$ and take the product–exponential adjunction

$$(- \times S) \;\dashv\; (S \to -).$$

Its two composites are, on the nose, the two structures the system runs on:

$$\underbrace{(S \to -) \circ (- \times S)}_{S \to (A \times S)} = \mathsf{State}\,S, \qquad \underbrace{(- \times S) \circ (S \to -)}_{(S \to A) \times S} = \mathsf{Store}\,S.$$

$$\mathbf{Set} \;\underset{S \to -}{\overset{- \times S}{\rightleftarrows}}\; \mathbf{Set} \qquad (\perp)$$

The inner simulation threads the world, `World` $\to$ (*flows*, `World`) — the **State monad** at $S =$ `World`. The outer sweep varies the parameters and reads out, (`Params` $\to$ `Result`, `Params`) — the **Store comonad** at $S =$ `Params`. These are the monad and the comonad induced by *the same adjunction*, instantiated at two different objects. The thing that runs the economy and the thing that tunes it are the two faces of product-and-exponential.

**Why a comonad, not a monad, for tuning.** A monad builds a computation from context you put *in* (sequencing effects); a comonad reads a value *out* of a context that surrounds it. Tuning is "extract one verdict from a neighbourhood of configurations" — a co-Kleisli, context-dependent extraction. The direction of information flow is opposite to the simulation's, so the structure dualises.

## 6 What the harness lets you do — and why the comonad makes it elegant

This is the payoff. Each capability below is enabled by a specific comonad operation, and in each case the comonad is what makes it a one-liner rather than a pile of bespoke bookkeeping.

| Capability | Operation | Why the structure makes it elegant |
|---|---|---|
| Counterfactual without drift | `peek` / `seek` | Asking "what if" and committing to a new baseline are distinct, typed acts; the classic mutate-and-forget-to-restore bug is unrepresentable. |
| The grid in one line | `experiment` | Sampling a shape of positions is a comonad primitive, not a bespoke loop with index bookkeeping. |
| Gradients for free | `extend` | The *same* operator that defines the grid defines the derivative; no separate differentiation engine. This is what surfaced `stapleReservePrice` > `firmFloat`. |
| Higher-order sensitivity | `extend` ∘ `extend` | Cross-elasticities (does one knob change another's influence?) compose, and coassociativity *guarantees* the result is coherent and order-independent. |
| Steepest tuning | `extend` | Having the local gradient field, you can step the knob with the best $\partial$(objective) — hill-climbing over the response surface is a fold over an `extend`. |
| Verified instrument | the laws | Determinism + the comonad laws mean the numbers the harness reports are sound, not merely plausible; the tuning tool is itself under test. |
| Model-agnostic reuse | `Store` | `store.ts` knows nothing about economics. Point the readout at any deterministic simulation and the entire tuning calculus transfers unchanged. |
| Principled architecture | the adjunction | Simulation and tuning are not two unrelated subsystems; they are the monad and comonad of one adjunction, so the design is forced, not improvised. |

**The throughline.** The comonad cleanly separates *the field of all configurations* from *the local computation you run over it.* Once that separation is a first-class type, every neighbourhood-aware analysis — gradients, Hessians, hill-climbing, what-if panels — becomes a short composition of `extract`, `peek` and `extend`, and the laws guarantee those compositions mean what they say. Without the comonad you would hand-roll configuration mutation, restoration, and neighbour bookkeeping by hand, and nothing would guarantee that a gradient-of-a-gradient is even coherent.

## 7 Honest caveats

- **One impure cell.** `params` is a module singleton, so the readout snapshots it, assigns the position, runs, and restores. That cell is quarantined and memoised; the comonad *above* it is pure and lawful. Its denotation is a deterministic `Params` $\to$ `Result`, and tabulating a deterministic function over the finite set of positions that `experiment` and `extend` touch does not change its meaning.
- **Secants, not tangents.** The knobs are integers; there is no $h \to 0$. The forward difference *is* the honest object, and the step size is part of the question, not an approximation error.
- **No optimiser yet.** The structure supports steepest-ascent tuning; the harness currently reports the gradient field but does not itself walk it.

## A Core listings

```
// store.ts -- the Store comonad, pure and lawful over a deterministic readout.
export function extract<S, A>(w: Store<S, A>): A { return w.peek(w.pos); }
export function peek<S, A>(s: S, w: Store<S, A>): A { return w.peek(s); }
export function seek<S, A>(s: S, w: Store<S, A>): Store<S, A> {
  return { peek: w.peek, pos: s };
}
export function map<S, A, B>(f: (a: A) => B, w: Store<S, A>): Store<S, B> {
  return { peek: (s) => f(w.peek(s)), pos: w.pos };
}
export function duplicate<S, A>(w: Store<S, A>): Store<S, Store<S, A>> {
  return { peek: (s) => ({ peek: w.peek, pos: s }), pos: w.pos };
}
export function extend<S, A, B>(k: (w: Store<S, A>) => B, w: Store<S, A>): Store<S,
    B> {
  return { peek: (s) => k({ peek: w.peek, pos: s }), pos: w.pos };
}
export function experiment<S, A>(shape: (s: S) => readonly S[], w: Store<S, A>): A
   [] {
  return shape(w.pos).map((s) => w.peek(s));
}
```

```
// store_test.ts -- the laws, checked by sampling the readout (stores hold
    functions).
Deno.test("extract then duplicate = id", () => {
  assert(storeEq(SAMPLE, extract(duplicate(w)), w, eqNum));
});
Deno.test("coassociativity", () => {
  const left  = duplicate(duplicate(w));
  const right = map(duplicate, duplicate(w));
  assert(storeEq(SAMPLE, left, right, eqMiddle));
});
Deno.test("extract . extend k = k", () => {
  const k = (v) => extract(v) + peek(v.pos + 1, v);
  assert(extract(extend(k, w)) === k(w));
});
```